



MASTER DE MATHÉMATIQUES
PARCOURS "Modélisation et Analyse Numérique"

PROGRAMMATION II - HAX011X - 2025/2026

PW1 - A CLASS list

Exercise 1. A template class list

By definition, an array in C++ (as well as other programming languages) must contain components that are all of the same type and size. However, in many applications, one needs to use sequences of objects that may differ in size from each other. For example, one may need to use sequences of vectors whose dimensions are not yet known at compilation-time. Thus, some flexible data-structures are needed.

In this exercise, we design a first flexible data-structure as a list class, which has an array of pointer-to-T objects. Although objects of type T may have different sizes (e.g., when T is a vector with dynamic memory), their addresses are just integer numbers that occupy a fixed amount of memory and, hence, can be placed in an array. During run-time, concrete objects of type T are placed in the addresses in the array.

The length of the list, that is, the number of pointers-to-T in the array in it, can also be determined dynamically at run-time: this is done by using an extra integer field to store the number of items.

Write this template list class with the following guidelines:

- (1) the list class contains two **protected** data fields: an **int** number_ to indicate the number of items and a T** item_ to store their addresses,
- (2) a first constructor that gets the value of number_ and allocate the corresponding array of pointer-to-T,
- (3) a second constructor that gets the value of number_, together with a T object to initialize the array of pointer-to-T (all the list values are initialized to T),
- (4) a copy constructor with the prototype list (list<T> const&),
- (5) an overloading of **operator=**,
- (6) a destructor to release the memory,
- (7) an overloading of **operator[]**(int i) to provide access to the value *(item[i]) pointed-at by the protected field item[i] in a *read-only* fashion,
- (8) an overloading of **operator()**(int i) to provide access to the value *(item[i]) pointed-at by the protected field item[i] in a *read-write* fashion,
- (9) a method number() to provide access to the protected field number_ in a *read-only* fashion, and a method number() to provide access to the protected field number_ in a *read-write* fashion.
- (10) a method item(int i) to provide access to the protected field item_[i] in a *read-only* fashion, and a method item(int i) to provide access to the protected field item_[i] in a *read-write* fashion,
- (11) an overloading of **operator<<** to pretty-print the list's items.

Test and assess your class (test all methods) in a main() program.

Exercise 2. Application I: Taylor expansion

- (1) let $f \in C^\infty(D_f, \mathbb{R})$ be a function of the independent variable x and at a given point $a \in D_f$, we aim at calculating the derivatives of f up to order n . But where should we store these calculated values ? In C, one is forced to use an array, but from OOP an array is not a meaningful object, and passing it to a function or returning it from a function is not very handy. The list object is more suitable for this purpose and this advantage of the list object over the standard array becomes essential in complex applications, where many functions are applied to objects returned from other functions. We consider

$$f(x) = \frac{1}{x},$$

such that we have

$$f^{(k)}(x) = -\frac{k}{x} f^{(k-1)}(x).$$

Define a function declared as follows

```
template<typename T>
const list<T> derive_inverse(T const& a, int n);
```

which returns the list of derivatives $(f^{(i)}(a))_{0 \leq i \leq n}$.

- (2) Let x be fixed, and let h be a small parameter. Assume that f has sufficiently many derivatives in the closed interval $[x, x + h]$. The Taylor-Lagrange expansion of order n at $x + h$ gives the value of the function at $x + h$ in terms of the values of the function and its derivatives at x , plus an error term that involves the $(n + 1)^{\text{th}}$ derivative at an intermediate point $x \leq \zeta \leq x + h$:

$$f(x + h) = \sum_{i=0}^n \frac{f^{(i)}(x)}{i!} h^i + \frac{f^{(n+1)}(\zeta)}{(n+1)!} h^{n+1}.$$

When f is sufficiently smooth, one can assume that the $(n + 1)^{\text{th}}$ derivative is bounded, so for sufficiently large n the error term is negligible. In this case, a good approximation to $f(x + h)$ is given by

$$f(x + h) \approx \sum_{i=0}^n \frac{f^{(i)}(x)}{i!} h^i.$$

In the problem of numerically computing this sum, the required informations about f and its derivatives at x are stored in a list<T> object, as described in the previous question, and passed to a function called Taylor() together with the given value of x . Hence, such function should be declared as follows:

```
template<class T>
const T Taylor(list<T> const& f, T const& h);
```

Note that in order to efficiently compute the "power" terms $\frac{h^i}{i!}$, we observe that

$$\frac{h^i}{i!} = \frac{h}{i} \frac{h^{i-1}}{(i-1)!}$$

Exercise 3. Application II: binomial object

Given an integer value M , we want to embed Pascal's triangle (which is populated with the binomial coefficients) in a `list` object, made of `dynamicVectors<T>` (this class is defined in a `dynamicVectors.hpp` file, which is provided on the course website).

More precisely, the new class is required to be of the `list<dynamicVector<T>>` type, but we want to have some new dedicated constructor to handle the particular **template** type in a suitable way. As a consequence, we publicly derive the new class called `binomial` from the aggregated `list<dynamicVector<T>>` class:

```
template<typename T>
class binomial : public list<dynamicVector<T>>{
```

The `binomial` class should have two **public** methods:

- (1) a constructor that get the total number of lines in the triangle, in order to create the correctly-sized `list` and populate it with binomial coefficients using the Pascal formula:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

- (2) an overloading of **operator()**(`int` n , `int` k) to ease the access to the value $\binom{n}{k}$ in a *read-only* fashion (hence this operator returns a `T`).

Test and assess your class (test all methods) in a `main()` program. Print some coefficients on screen for validation.

Exercise 4. Application III: Derivatives of a product

We consider the following computation problem: assume that the derivatives of the functions $x \mapsto f(x)$ and $x \mapsto g(x)$ at a given point x up to and including order n are available. Compute the n^{th} derivative of the product fg at x , denoted by $(fg)^{(n)}(x)$ as follows:

$$(fg)^{(n)}(x) = \sum_{i=0}^n \binom{n}{i} f^{(i)}(x) g^{(n-i)}(x).$$

Define a function `derive_product()` that takes as input two `list` objects that contain the derivatives of f and g up to and including order n at the fixed point x , together with the value of n , and returns the n^{th} derivative of fg at x (only the n^{th} derivative).